ProEm: The Branch Unit

`src/core/frame_chain/branch_unit.sv`

Paul Vincent Bezzina

17 October 2026

Abstract

The branch unit is what lets a frame react to itself. Without it a frame is a fixed list of blocks run in one direction; with it, a frame can read a field and then turn into a write (an I^2C target answering a read), stop when a device says WAIT or FAULT (SWD), give up the bus the moment it loses arbitration (CAN), or act on its own CRC check. The module is short, about 100 lines of logic and roughly 390 cells per chain, and most of its ports are plumbing. What is worth explaining is its timing: when a rule is allowed to look, how the chain is held still for one clock while it decides, which outputs are immediate and which are registered, and why each rule fires only once per frame. Names in `typewriter` are the signal names in the RTL.

Contents

1 Where it sits **1**

2 Ports **2**

3 Configuration **2**

4 How one rule decides **2**

4.1 Source 0: a block's value . . . 2

4.2 Source 1: an event on the line . . . 3

4.3 From a firing rule to an action . . . 3

5 Two rules, one answer **3**

6 One shot per frame **3**

7 Outputs and timing **4**

8 What the chain does with it **5**

8.1 `line_watch` . . . 5

9 Worked examples **5**

9.1 I^2C target: direction from the R/W bit . . . 5

9.2 SWD-style acknowledge . . . 5

9.3 CAN-style arbitration . . . 5

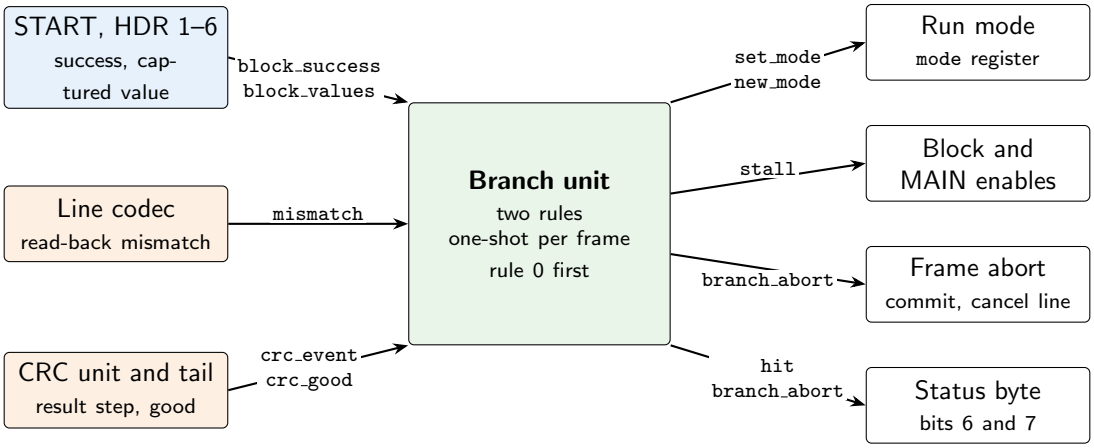
9.4 Acting on the CRC . . . 5

10 Verification **6**

11 Signal glossary **6**

1 Where it sits

Each frame chain has one branch unit, between its blocks and the chain's control logic. It watches; it never touches the line itself. Everything it does goes through four effects in `frame_chain`: the run mode, a one-clock stall, a frame abort, and two status bits.



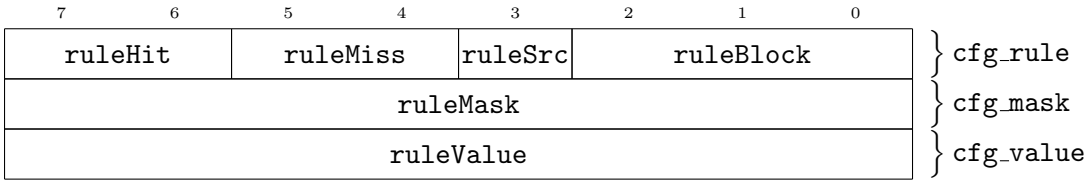
A sixth output, `line_watch`, is not an action at all: it tells the chain to keep a write frame open until its last bit has left the pin (Section 8.1).

2 Ports

Port	Dir	Meaning
`cfg_we`, `cfg_sel`	in	Configuration write; `cfg_sel` 8 is rule 0, 9 is rule 1 (`BRANCH 0` and `BRANCH 1`).
`cfg_rule/mask/value`	in	The three operand bytes of `BRANCH`.
`block_success`	in	One bit per block, START to `HDR 6`; high from the clock a block finishes until the frame ends.
`block_values`	in	The eight-bit value of each block: what a read block received (or compared), what a write block sent.
`mismatch`	in	From the line codec: the pad did not read back the level the chip drove, at this bit's check point.
`crc_event`, `crc_good`	in	The tail is pushing the CRC result entry this clock, and whether the check passed.
`in_frame`	in	START has succeeded (`block_success[0]`); rules only look inside a frame.
`frame_reset`	in	The frame is ending (complete, abort, timeout, halt).
`stall`	out	Hold every block and MAIN this clock.
`set_mode`, `new_mode`	out	Change the run mode to read (1) or write (2).
`hit`	out	A rule took its `HIT` action (registered).
`branch_abort`	out	Abort the frame (registered).
`line_watch`	out	A read-back rule is set: hold write frames open until the line is idle.

3 Configuration

Each rule is five small registers, loaded from one `BRANCH` instruction:



- `ruleOn` is set when the first byte is not zero, so `BRANCH n` with byte 1 = `00` turns a rule off. (A rule whose actions are both KEEP is still on; that is how `line_watch` is set without changing anything else.)
- Writing a rule also clears its `fired` flag, so a rule rewritten mid-frame can act again in that frame.
- The actions are `00` KEEP, `01` READ, `10` WRITE and `11` ABORT, the same for `HIT` and `MISS`.

4 How one rule decides

Both rules are the same logic, written as a loop over `r` = 0, 1. A rule has two kinds of source, picked by `ruleSrc`.

4.1 Source 0: a block's value

```
watched[r]   = block_values[ruleBlock[r]]
matched[r]   = ((watched[r] ^ ruleValue[r]) & ruleMask[r]) == 0
fireBlock[r] = ruleOn[r] && !ruleSrc[r] && !fired[r] && in_frame
               && ruleBlock[r] < 7 && block_success[ruleBlock[r]]
```

`watched` is an eight-way multiplexer over the block values. The match is a masked compare: every bit set in `MASK` must equal the same bit of `VALUE`; bits outside the mask are ignored, so `MASK` = `00` always matches and `MASK` = `01` tests one bit. The rule *fires* on the first clock its block shows success. `BLOCK` = 7 never fires (there is no block 7; MAIN cannot be watched).

4.2 Source 1: an event on the line

```
lineEvent[r] = (ruleBlock[r] == 0) ? mismatch
                                   : (ruleBlock[r] == 1) && crc_event
lineHit[r]   = (ruleBlock[r] == 0) || crc_good
fireLine[r]  = ruleOn[r] && ruleSrc[r] && !fired[r] && in_frame && lineEvent[r]
```

- `BLOCK` = 0, read-back: fires on the first bit where the pad disagrees with what the chip drove. A mismatch always counts as a hit, so only `HIT` is used. This is CAN arbitration: drive recessive, see dominant, you have lost.
- `BLOCK` = 1, CRC: fires when the tail emits the CRC result. `HIT` if the CRC was good, `MISS` if it was bad.
- Any other `BLOCK` with `SRC` = 1 never fires.

4.3 From a firing rule to an action

```
fire[r]    = fireBlock[r] || fireLine[r]
tookHit[r] = (fireLine[r] && lineHit[r]) || (fireBlock[r] && matched[r])
action[r]  = (fireLine[r] ? lineHit[r] : matched[r]) ? ruleHit[r] : ruleMiss[r]
```

`tookHit` feeds the status bit; `action` is what the rule asks for this clock. A rule that does not fire still has an `action` value, but it is ignored.

5 Two rules, one answer

```
chosen = KEEP
if      (fire[0] && action[0] != KEEP) chosen = action[0]
else if (fire[1] && action[1] != KEEP) chosen = action[1]
```

Rule 0 wins only if it actually asks for something. If rule 0 fires with KEEP and rule 1 fires with READ in the same clock, the frame switches to read. Both rules still count for the status: `hit` is the OR of both `tookHit` bits.

In practice the two rules rarely fire together. They usually watch different blocks (for example, rule 0 on the address, rule 1 on a command byte), or one watches a block and the other the line.

6 One shot per frame

Each rule has a `fired` flag. It is set in the clock the rule fires and cleared by `frame_reset` (or by rewriting the rule). While it is set the rule cannot fire again. Two reasons:

- `block_success` stays high for the rest of the frame once a block has finished. Without the flag a block rule would fire on every clock after that.
- A read-back rule should act on the first mismatch, not on every later bit of a frame it has already given up.

7 Outputs and timing

Some outputs are combinational and act in the same clock the rule fires; others are registered and act one clock later. The split is deliberate.

Output	When	Why
`stall`	same clock, and the clock after an abort	`|fireBlock || internal_abort`. Freeze the chain while a block rule decides, so the next block cannot start in the old mode.
`set_mode`, `new_mode`	same clock	The chain loads `mode` at the end of this clock, so the next block starts in the new mode.
`branch_abort`	one clock later (`internal_abort`)	Abort fans out to the frame buffer, the codec and the status; registering it keeps the path from the block values to all of them short.
`hit`	one clock later (`internal_hit`)	Only feeds a status flag.

Both registered outputs are suppressed if `frame_reset` arrives in the same clock, so a rule firing at the very end of a frame cannot leak into the next one.

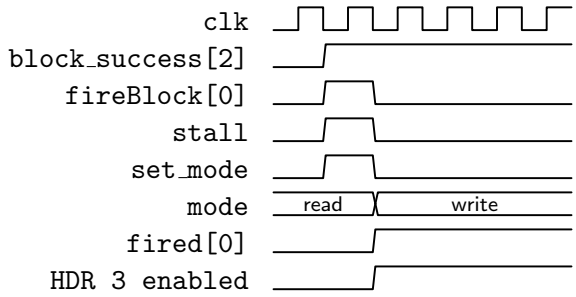


Figure 1: A rule on `HDR 2` that switches to write: it fires in the first clock `HDR 2` shows success, the chain stalls for that clock, and `HDR 3` starts in the next clock with the new mode already in place.

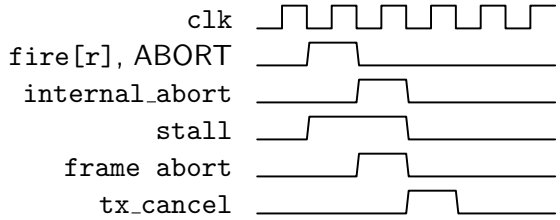


Figure 2: An abort: the frame aborts one clock after the rule fires, the stall covers that clock too, and the line codec drops its queued bits one clock after that (`tx_cancel` is registered in the chain).

A line rule (read-back or CRC) does not stall the chain: the bit is already on the wire and the chain cannot take it back. A read-back abort therefore stops the line about two clocks after the mismatch check, well inside the bit; with ASYNC timing the check is mid-bit, so the chip stops driving within the bit it lost.

The cost of the stall. One system clock per block rule per frame. Every standard ProEm runs has at least two system clocks per bit, and the stall falls between blocks, so no bit is lost.

8 What the chain does with it

- **Mode.** `mode` is loaded from `RUN`'s mode when the chain is armed, from `new_mode` on `set_mode`, and back to the `RUN` mode at every frame end. The mode decides which blocks run (`GRP` = `01` only when reading, `10` only when writing) and which way a MAIN with `DIR` = follow goes.
- **Enables.** `stall` is ANDed out of every header's enable and MAIN's enable.
- **Abort.** `branch_abort` is one of the chain's abort sources, but a gentler one. It *commits* what has been captured so far instead of rolling it back, so the host sees the value that caused the abort (an SWD acknowledge, say). It also cancels whatever the line codec still has queued (`cancel_now`), and the frame ends with its status entry as usual.
- **Status.** Bit 6 of the frame's status byte is set if any rule took its `HIT` action during the frame (`hit_seen`); bit 7 if a branch aborted it. An abort on `HIT` therefore reports `0xC2`, on `MISS` `0x82`.

8.1 `line_watch`

`line_watch` is high while either rule is on with `SRC` = 1 and `BLOCK` = 0. A write frame normally completes as soon as MAIN has handed its last entry to the codec, a few bit times before that bit leaves the pin. A late mismatch would then arrive after the frame had ended and be lost. With `line_watch` the chain waits (`finish_wait`) until the codec reports the line free before completing, so the read-back rule sees every bit.

The same signal is useful on its own. A rule `F0 08 00 00` (both actions KEEP, read-back) changes nothing but keeps write frames open until the line is idle. The K-Line and SDI-12 tests use it so that the next frame is not armed while the last stop bit is still being sent.

9 Worked examples

9.1 I²C target: direction from the R/W bit

`HDR 2` captures the R/W bit with `ANY`. Rule `F0 92 01 01`: `HIT` = WRITE, `MISS` = READ, `SRC` = 0, `BLOCK` = 2, `MASK` = `01`, `VALUE` = `01`. If the controller asked to read (R/W = 1), the chip switches to write and MAIN (`DIR` = follow) sends the loaded `DATA`; otherwise it reads the controller's bytes. The status is `41` for a read (bit 6: the rule hit) and `01` for a write.

9.2 SWD-style acknowledge

`HDR 2` reads the three-bit acknowledge. Rule `HIT` = READ, `MISS` = ABORT, `BLOCK` = 2, `MASK` = `07`, `VALUE` = `01`. OK continues into the data phase; WAIT or FAULT aborts with `0x82`, and the host has already received the acknowledge value because a branch abort commits what was captured.

9.3 CAN-style arbitration

The chip drives an open-drain line and reads it back. Rule `HIT` = ABORT, `SRC` = 1, `BLOCK` = 0. On the first bit where the chip sends recessive but reads dominant, the frame aborts with `0xC2` and the chip stops driving within that bit; the winning node's frame continues untouched.

9.4 Acting on the CRC

Rule `SRC` = 1, `BLOCK` = 1 fires when the tail emits the CRC result: `HIT` if the check passed, `MISS` if not. With `MISS` = ABORT a bad frame is reported as aborted (`0x82`) instead of complete.

10 Verification

Check	What it covers
`formal/branch_unit.sby`	Unbounded proof against a model of the `BRANCH` rules: masked compare, hit and miss actions, read-back and CRC sources, rule 0 before rule 1, each rule acting at most once per frame, and the timing of `stall`, `hit`, `branch_abort` and `line_watch`.
`t_branch`	SWD-style acknowledge (OK continues, WAIT and FAULT abort); CAN-style arbitration won and lost.
`t_i2c_target`	Direction from the R/W bit, with repeated START.
`t_modes`	Rule 1 on its own value, rule 0 on another, in the same program.
`t_random_branch`	600 random programs: a rule on a captured header with random mask, value, actions and starting mode, then a read-only and a write-only header; checks which header runs, the data returned and the status byte.
`t_standards`	Read-back rules used only for `line_watch` (K-Line, SDI-12).

The one combination without a pin-level test is a rule on the CRC result (`SRC` = 1, `BLOCK` = 1). The formal proof covers it inside the module, and the CRC tests cover the `crc_event` and `crc_good` it relies on, but no test yet runs the two together through the whole chip.

11 Signal glossary

Signal	Meaning
`ruleHit`, `ruleMiss`	Action on a match and on a miss.
`ruleSrc`, `ruleBlock`	What the rule watches: a block (0) or a line event (1), and which one.
`ruleMask`, `ruleValue`	The masked compare for block rules.
`ruleOn`	The rule is configured (byte 1 not zero).
`watched`	The value of the block the rule watches.
`matched`	The watched value passes the masked compare.
`fireBlock`, `fireLine`	The rule acts this clock, from a block or from the line.
`lineEvent`, `lineHit`	The line event happened, and whether it counts as a hit.
`fire`, `action`, `tookHit`	Per rule: acting, what it asks for, and whether it took `HIT`.
`chosen`	The action that wins this clock (rule 0 first, KEEP does not win).
`fired`	The rule has acted in this frame.
`internal_hit`, `internal_abort`	Registered `hit` and `branch_abort`.